LAPORAN TEKNIS UJIAN TENGAH SEMESTER

SISTEM BASIS DATA TERDISTRIBUSI

BAGIAN III – TUGAS PROYEK TERINTEGRASI

Implementasi Basis Data Relasional Terdistribusi, Single Sign-On, dan Inferensi Model dengan Docker dan Kubernetes

Disusun oleh:

Marcel Immanuel

NIM: 20240801117

Mata Kuliah: Sistem Basis Data Terdistribusi

Dosen: Imam Sutanto, S.Kom., M.Kom.

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS ESA UNGGUL
TAHUN AJARAN 2026/2027

1. Ringkasan Proyek dan Status Implementasi

Proyek ini membuat demonstrasi layanan prediksi risiko transaksi menggunakan data sintetis. Pengguna mendapatkan token login dari Keycloak, kemudian mengirim data transaksi ke FastAPI. API memeriksa token dengan kunci JWKS, menghitung prediksi model Logistic Regression, menyimpan hasil ke PostgreSQL Primary dan menyediakan endpoint riwayat yang membaca PostgreSQL Read-Replica. Container dikelola dengan Docker Compose untuk uji lokal dan manifest Kubernetes untuk orkestrasi.

Tujuan utama proyek adalah membuktikan autentikasi SSO, pemisahan operasi tulis/baca pada database relasional terdistribusi, transparansi alamat layanan, dan persistensi data saat Pod database dibuat ulang. Tidak terdapat antarmuka website khusus; interaksi dapat dilakukan melalui endpoint API dan Terminal.

Batasan: dataset dan kata sandi yang dikemas pada contoh ini bersifat demonstrasi. Model tidak untuk menilai transaksi atau nasabah nyata.

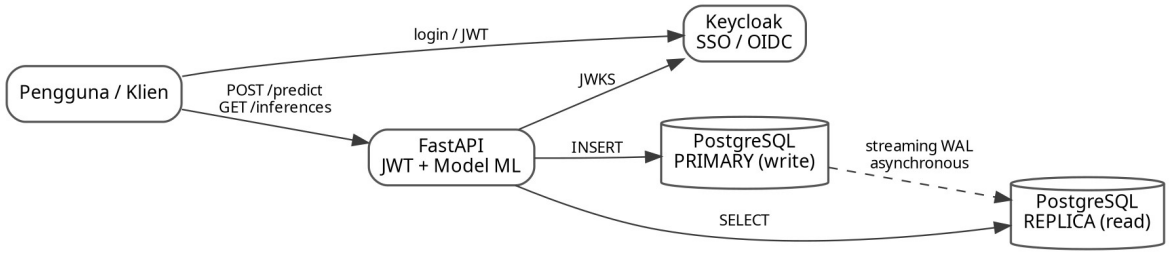
STATUS VERIFIKASI: Source code dan konfigurasi proyek telah tersedia. Dalam pemeriksaan statis, berkas Python lolos compileall; compose.yaml dan sembilan manifest Kubernetes dapat di-parse; konfigurasi realm Keycloak adalah JSON valid. Pemeriksaan ini tidak membuktikan aplikasi berjalan. Pengujian Docker, login JWT end-to-end, replikasi WAL, penghapusan Pod, dan Ingress belum dibuktikan dengan screenshot aktual. Kolom hasil pengujian sengaja dibiarkan untuk diisi setelah pelaksanaan.

2. Pemetaan Rubrik Penilaian Dosen

Komponen dan bobot	Berkas / rancangan yang tersedia	Bukti yang masih harus dibuat
Arsitektur & Docker Setup – 20%	Dockerfile multi-stage; compose.yaml; empat layanan API, Keycloak, Primary, Replica	Screenshot docker compose ps, API health, proses build dan log tanpa error
Kubernetes & Storage Persistence – 25%	StatefulSet Primary/Replica; PVC; Service; ConfigMap; Secret; Ingress manifest	Screenshot Pod/PVC Bound, data sebelum dan setelah hapus Pod, Ingress benar-benar dapat diakses
Integrasi SSO – 20%	Realm sbdt; akun demo; FastAPI memvalidasi JWT RS256, issuer, audience, exp via JWKS	Screenshot token diperoleh (disamarkan), akses berhasil dan akses tanpa token ditolak 401
Model & Replikasi Relasional – 20%	Logistic Regression sintetis; POST /predict ke Primary; GET /inferences dari Replica	Screenshot POST 201, GET, pg_is_in_recovery, pg_stat_replication=streaming
Laporan & Showcase Git – 15%	Diagram, README, skrip uji, laporan PDF, source code dalam ZIP	URL repositori Git asli, README, screenshot pengujian, laporan final PDF

Catatan penilaian: keberadaan file konfigurasi saja belum membuktikan layanan berfungsi. Rubrik menekankan verifikasi operasional; karena itu lima komponen di atas memerlukan bukti nyata. Tidak ada estimasi nilai yang diklaim pada tahap ini.

3. Arsitektur dan Alur Data



Gambar 1. Rancangan hubungan pengguna, Keycloak, FastAPI, PostgreSQL Primary, dan Read-Replica.

1. Pengguna meminta akses melalui Keycloak pada realm sbdt dan menerima JWT access token.
2. Klien mengirim POST /predict membawa Authorization: Bearer <token>.
3. FastAPI mengecek tanda tangan, issuer, audience, dan masa berlaku JWT menggunakan JWKS.
4. Model Logistic Regression menghitung peluang risiko dari data transaksi sintetis.
5. API menjalankan INSERT ke Primary; replikasi streaming WAL meneruskan perubahan ke Replica.
6. GET /inferences menjalankan SELECT pada Read-Replica. Replikasi asynchronous dapat menimbulkan jeda pembacaan.

4. Arsitektur & Docker Setup (Bobot 20%)

4.1. Dockerfile multi-stage dan lapisan build

Tahap builder menggunakan python:3.11-slim, memasang dependensi sebagai wheel, dan melatih model demonstrasi. Tahap runtime menyalin dependensi serta artefak model, menjalankan Uvicorn pada port 8000, dan menggunakan user non-root appuser. Pemisahan tahap ini ditujukan menjaga image runtime berisi kebutuhan untuk menjalankan aplikasi saja.

4.2. Empat layanan Compose

Layanan	Fungsi	Port lokal / jaringan
postgres-primary	Sumber tulis dan WAL replikasi	Internal 5432
postgres-replica	Standby read-replica berbasis pg_basebackup	Internal 5432
keycloak	SSO OIDC dan penerbit JWT	http://localhost:8080
api	FastAPI, validasi JWT, inferensi, read-write routing	http://localhost:8000

Setiap database menggunakan volume tersendiri (primary_data dan replica_data). Nama layanan pada Compose dipakai sebagai hostname sehingga API tidak perlu mencantumkan alamat IP container. Keycloak start-dev pada contoh belum disiapkan untuk produksi.

4.3. Prosedur pengujian Docker

```
docker --version
docker compose version
docker compose up -d --build
docker compose ps
bash scripts/smoke_test.sh
bash scripts/probe_replication.sh
```

Kriteria lulus: empat container berjalan, GET /health mengembalikan status ok, POST /predict mengembalikan HTTP 201, GET /inferences menampilkan data, dan primary/replica mempunyai peran berbeda. Lampirkan screenshot docker compose ps dan keluaran pengujian.

5. Kubernetes & Storage Persistence (Bobot 25%)

5.1. Pemetaan resource Kubernetes

Resource	Implementasi	Peran
Namespace	sbdt-uts	Memisahkan resource proyek
StatefulSet + PVC	postgres-primary; postgres-replica; 1Gi per Pod	Identitas stabil dan volume database
Service / Headless Service	postgres-primary, postgres-replica, keycloak, api	DNS internal tanpa IP Pod tetap
ConfigMap	pg-init, replica-init, keycloak-realm, app-config	Menyimpan konfigurasi non-rahasia
Secret	sbdt-secrets	Menyimpan password contoh (bukan production secret)
Deployment	keycloak, api	Menjalankan layanan stateless/demonstrasi
Ingress	auth.lokal dan api.lokal	Rencana akses dengan hostname

Deployment PostgreSQL menggunakan StatefulSet (masing-masing satu Pod), dengan volumeClaimTemplates yang menghasilkan PersistentVolumeClaim. Saat Pod database dihapus lalu dibuat ulang, PVC seharusnya tetap terpasang sehingga data pada disk dapat dibuka kembali. Ini adalah bukti persistence, bukan otomatis failover.

5.2. Langkah pengujian manifest dan PVC

```
docker compose down
docker build -t sbdt-api:local .
kind create cluster --name sbdt-uts
kind load docker-image sbdt-api:local --name sbdt-uts
kubectl apply -f k8s/00-namespace.yaml
kubectl apply -f k8s/
kubectl -n sbdt-uts get pods,svc,pvc,ingress
```

Buka dua jendela Terminal dan jalankan port-forward Keycloak pada 8080 serta API pada 8000. Setelah smoke test memasukkan sekurang-kurangnya satu rekaman, catat hasil SELECT count(*) pada Primary, hapus Pod postgres-primary-0, lalu periksa jumlah record setelah Pod kembali Ready.

```
kubectl -n sbdt-uts get pvc
kubectl -n sbdt-uts exec postgres-primary-0 -- psql -U sbdt -d sbdt -c 'SELECT count(*) FROM inference_history;'
kubectl -n sbdt-uts delete pod postgres-primary-0
kubectl -n sbdt-uts rollout status statefulset/postgres-primary
kubectl -n sbdt-uts exec postgres-primary-0 -- psql -U sbdt -d sbdt -c 'SELECT count(*) FROM inference_history;'
```

Kriteria lulus: Pod kembali Running/Ready, PVC masih Bound, dan record yang telah disimpan sebelum penghapusan tetap ada. Catat angka sebelum dan sesudah sebagai bukti. Sistem contoh belum memiliki automatic primary promotion ketika Primary gagal.

5.3. Keterbatasan Ingress yang perlu dituntaskan

File k8s/08-ingress.yaml sudah mendefinisikan auth.lokal dan api.lokal serta ingressClassName: nginx. Akan tetapi, proyek tidak menyertakan instalasi controller Ingress; membuat objek Ingress saja tidak membuktikan routing hostname berfungsi. Sebelum menguji, pilih dan pasang controller yang masih dikelola, sesuaikan ingressClassName apabila berbeda, lalu uji kedua hostname. Community ingress-nginx lama telah dihentikan pemeliharaannya pada Maret 2026, sehingga tidak direkomendasikan untuk pemasangan baru.

6. Integrasi Single Sign-On / SSO (Bobot 20%)

Keycloak menerbitkan access token dari realm sbdt kepada pengguna demo. Backend FastAPI mengunduh kunci publik JWKS melalui endpoint Keycloak internal untuk memvalidasi JWT. Backend

memeriksa algoritma RS256, signature, expiration (exp), waktu terbit (iat), issuer (iss), subject (sub), dan audience api-service. Token yang tidak tersedia atau tidak valid harus memperoleh HTTP 401.

Skenario uji: (1) mendapatkan token uji; (2) POST /predict dengan Bearer token valid dan mendapat HTTP 201; (3) mengulangi POST tanpa Authorization dan mendapat HTTP 401; (4) mengirim token rusak dan mendapat HTTP 401. Jangan pernah menampilkan kata sandi atau token JWT lengkap dalam screenshot publik.

Akun demo marcel dan password demonstrasi tersedia dalam README proyek. Direct access grant digunakan untuk smoke test lokal, sedangkan penggunaan nyata lebih baik memakai OIDC Authorization Code + PKCE dengan HTTPS.

7. Integrasi Model & Replikasi Relasional (Bobot 20%)

Model scikit-learn Logistic Regression dilatih dari 600 data sintetis dengan fitur transaction_amount, transactions_24h, dan new_device. Output berupa risk_probability dan risk_label (tinggi/rendah). Model hanya contoh untuk membuktikan integrasi inference engine dengan database relasional.

Skema basis data inference_history menyimpan id, username, transaction_amount, transactions_24h, new_device, risk_probability, risk_label, created_at. Endpoint POST /predict menyimpan record pada hostname postgres-primary. Endpoint GET /inferences membaca hingga 30 hasil pengguna terkait dari hostname postgres-replica.

Bukti routing terdiri atas dua tingkat. Header respons X-Write-Node dan X-Read-Node menunjukkan keputusan rute oleh backend. Bukti sisi PostgreSQL lebih kuat: SELECT pg_is_in_recovery() harus false pada Primary, true pada Replica, serta pg_stat_replication pada Primary idealnya menunjukkan state streaming.

```
bash scripts/smoke_test.sh
bash scripts/probe_replication.sh
```

Kriteria lulus: respons prediksi benar-benar tersimpan dan muncul dari Replica setelah replikasi; hasil database dapat dibandingkan dengan payload yang dikirim. Karena replikasi asynchronous, uji pembacaan boleh diulang setelah beberapa detik jika data baru belum muncul.

8. Laporan, Showcase Git, dan Bukti Uji (Bobot 15%)

Struktur repositori mencakup Dockerfile, compose.yaml, requirements.txt, app/, db/, keycloak/, k8s/, scripts/, tests/, docs/, serta README.md. Setelah uji sukses, simpan screenshot terpilih pada docs/bukti/ dan tambahkan ringkasan hasil ke laporan. Unggah folder proyek ke repositori GitHub yang benar-benar dibuat dan pastikan tautannya dapat dibuka dosen.

URL repositori (diisi setelah berhasil push): _____

Pengujian dilakukan pada tanggal (diisi): _____ Perangkat: _____

8.1. Matriks pengujian yang perlu diisi

ID / Bukti	Langkah atau perintah	Ekspektasi	Hasil aktual
U01 Docker	docker compose ps	4 service tersedia/berjalan	Belum diuji
U02 API	GET /health	HTTP 200 status ok	Belum diuji
U03 SSO	Ambil token; POST token valid/tanpa token	201 untuk valid; 401 tanpa token	Belum diuji
U04 Write/Read	POST /predict; GET /inferences	Riwayat dibaca dari Replica	Belum diuji
U05 WAL	bash scripts/probe_replication.sh	Primary f; Replica t; streaming	Belum diuji
U06 K8s	kubectl get pods,svc,pvc	Pod Ready, PVC Bound	Belum diuji
U07 Persistence	DELETE Pod lalu SELECT count(*)	Record tetap tersimpan	Belum diuji
U08 Ingress	Uji auth.lokal & api.lokal via controller	Keduanya dirutekan	Belum diuji
U09 Git	Buka URL repository	File proyek dapat diakses	Belum diuji

Setelah pengujian, ganti status Belum diuji dengan LULUS atau GAGAL beserta catatan singkat. Screenshot harus berasal dari hasil komputer sendiri; jangan membuat bukti semu.

9. Panduan Operasional untuk Pengguna Mac Pemula

9.1. Persiapan perangkat lunak

1. Unduh Docker Desktop for Mac with Apple silicon dari https://docs.docker.com/desktop/setup/install/mac-install/ dan pindahkan Docker.app ke Applications.

2. Buka Docker Desktop sampai engine siap. Pengguna macOS tidak perlu menginstal PostgreSQL, Python server, ataupun Keycloak secara manual karena sudah dikemas di image container.

3. Ekstrak ZIP proyek, buka Terminal (Command + Space → Terminal), lalu masuk ke direktori proyek menggunakan perintah cd.

4. Periksa Docker: docker --version dan docker compose version. Jika command not found, pastikan Docker Desktop sudah dibuka dan instalasi CLI selesai.

5. Jalankan docker compose up -d --build, lalu docker compose ps. Buka http://localhost:8000/docs dan http://localhost:8080.

6. Jalankan bash scripts/smoke_test.sh dan bash scripts/probe_replication.sh. Simpan screenshot hasil yang tidak memuat token atau password.

7. Sesudah uji Docker, hentikan container menggunakan docker compose down (tanpa -v agar volume tetap tersimpan).

8. Untuk Kubernetes, pasang kind dan kubectl (misalnya brew install kind kubectl apabila Homebrew sudah ada), lalu ikuti perintah pada Bab 5.

9. Setelah semua pengujian, lengkapi matriks bukti dan PDF final, unggah proyek ke GitHub, lalu salin URL repository.

9.2. Perintah pertama yang dijalankan di Terminal

Jika folder hasil ekstraksi disimpan langsung di Downloads:

```
cd ~/Downloads/SBDT_Proyek_Bagian_III_Marcel_Immanuel
docker --version
docker compose version
docker compose up -d --build
docker compose ps
```

Tips: apabila folder tidak ada di Downloads, ketik cd di Terminal diikuti satu spasi, seret folder proyek dari Finder ke jendela Terminal, lalu tekan Enter. Dengan cara ini path folder akan terisi otomatis.

9.3. Catatan kapasitas MacBook Neo

MacBook Neo memiliki RAM 8 GB. Docker Desktop dapat dipasang, tetapi Keycloak, dua PostgreSQL, API, serta cluster Kubernetes sekaligus berpotensi menghabiskan banyak memori. Jalankan Docker Compose dan Kubernetes bergantian, tutup aplikasi berat, dan jangan menjalankan docker compose up serta kind bersamaan jika komputer melambat. Jika cluster terus gagal karena memori, gunakan PC/laboratorium dengan RAM lebih besar untuk pengujian Kubernetes.

10. Kendala, Keamanan, dan Kesimpulan

- Docker Desktop atau kubectl belum siap: cek versi CLI dan pastikan Docker engine berjalan.
- Docker Compose berstatus restarting/unhealthy: tinjau docker compose logs --tail=80, khususnya keycloak, postgres-primary, postgres-replica, dan api.
- Replica belum streaming: cek kredensial replikasi dan skrip pg_basebackup. Jangan menghapus volume yang berisi data penting.
- Pod Pending dan PVC tidak Bound: cek StorageClass dengan kubectl get storageclass, kapasitas disk, dan kubectl describe pvc.

- JWT ditolak 401: cek issuer dan audience sesuai lingkungan Docker/Kind; token Compose tidak selalu cocok dengan issuer cluster Kubernetes.
- Ingress tidak dapat diakses: manifest tanpa controller aktif tidak cukup. Konfigurasi routing hostname dan DNS/hosts harus diselesaikan.
- Password contoh dalam ZIP bukan rahasia produksi. Jangan memublikasikan token, password asli, file .env pribadi, atau data transaksi nyata.

Kesimpulan sementara: rancangan dan kode awal proyek telah mencakup lima kategori rubrik dalam bentuk berkas dan arsitektur. Kelengkapan penilaian masih bergantung pada keberhasilan deployment lokal, verifikasi SSO, pembuktian streaming replica, persistensi Pod/PVC, akses Ingress, screenshot yang dapat diaudit, serta URL Git yang valid. Oleh sebab itu, laporan ini adalah dokumen teknis siap dilengkapi hasil pengujian, bukan klaim implementasi yang telah lolos seluruh pemeriksaan.

Referensi Teknis

Docker. Install Docker Desktop on Mac. https://docs.docker.com/desktop/setup/install/mac-install/

Docker. Kubernetes in Docker Desktop. https://docs.docker.com/desktop/use-desktop/kubernetes/

Kubernetes. StatefulSets. https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/

Kubernetes. Ingress. https://kubernetes.io/docs/concepts/services-networking/ingress/

Kubernetes. Ingress NGINX retirement (2025). https://kubernetes.io/blog/2025/11/11/ingress-nginx-retirement/

kind. Quick Start. https://kind.sigs.k8s.io/docs/user/quick-start/

Keycloak. Getting started on Docker. https://www.keycloak.org/getting-started/getting-started-docker

PostgreSQL 16. pg_basebackup. https://www.postgresql.org/docs/16/app-pgbasebackup.html

PostgreSQL 16. Replication settings. https://www.postgresql.org/docs/16/runtime-config-replication.html